

Impacto de la Gestión de Procesos y Threads en la Capacidad de Respuesta de un Sistema Operativo de Tiempo Real

Arelys Ajila, Isaac Cartuche, Isauro Rivera, Josue Azansa, David Toledo
Carrera de Ingeniería en Computación
Universidad Nacional de Loja

Resumen—Los Sistemas Operativos de Tiempo Real (RTOS, *Real-Time Operating Systems*) constituyen el núcleo de múltiples aplicaciones críticas en sectores como la automatización industrial, la robótica, los sistemas aeroespaciales, la industria automotriz y el Internet de las Cosas (IoT). En estos entornos, la capacidad de respuesta depende directamente de la forma en que el sistema administra procesos, tareas y threads, así como de la eficiencia de los algoritmos de planificación y sincronización implementados. El presente ensayo analiza el impacto de la gestión de procesos y threads sobre la capacidad de respuesta de un RTOS, examinando aspectos como la planificación, la latencia, el cambio de contexto, la comunicación entre tareas, el uso de memoria, la resiliencia ante fallos y las tendencias emergentes en sistemas embebidos modernos. A partir de la revisión de diversos estudios académicos y comparaciones experimentales entre diferentes RTOS, se demuestra que una gestión eficiente de procesos y threads permite minimizar la latencia, reducir el jitter, mejorar la utilización de recursos y garantizar el cumplimiento de restricciones temporales estrictas. Los resultados evidencian que la elección del algoritmo de planificación, la implementación de mecanismos de sincronización adecuados y la optimización del uso de memoria son factores determinantes para lograr sistemas de tiempo real confiables, predecibles y eficientes en entornos de alta exigencia computacional.

Index Terms—RTOS, procesos, threads, planificación, latencia, cambio de contexto, tiempo real, sistemas embebidos, IoT, sincronización, jitter, determinismo.

I. INTRODUCCIÓN

Los Sistemas Operativos de Tiempo Real fueron desarrollados para garantizar que las tareas críticas se ejecuten dentro de límites temporales estrictos y predecibles. A diferencia de los sistemas operativos de propósito general, donde el rendimiento promedio constituye el objetivo principal, en un RTOS la característica fundamental es la capacidad de respuesta determinista frente a eventos externos. Esto implica que la correcta administración de procesos y threads tiene una influencia directa sobre el comportamiento temporal del sistema y sobre el cumplimiento de los plazos establecidos para cada tarea [6], [9].

La gestión de procesos y threads engloba mecanismos como la creación, destrucción, sincronización, comunicación y planificación de tareas. Estos mecanismos determinan la forma en que los recursos del procesador son distribuidos entre múltiples actividades concurrentes. Cuando la administración de threads es ineficiente, pueden producirse retrasos, inversión de prioridades, pérdidas de datos o incumplimiento de restric-

ciones temporales, afectando negativamente la capacidad de respuesta global del sistema [2], [3].

Históricamente, los primeros RTOS surgieron en la década de 1960 como respuesta a la necesidad de controlar sistemas de misión crítica en aplicaciones militares y aeroespaciales. Con el paso del tiempo, su uso se extendió hacia la industria automotriz, la medicina, las telecomunicaciones y, más recientemente, hacia el ecosistema del Internet de las Cosas. Esta evolución ha traído consigo nuevos desafíos relacionados con la gestión eficiente de múltiples tareas concurrentes en plataformas con recursos limitados [6], [7].

Actualmente, el crecimiento de aplicaciones complejas como vehículos autónomos, sistemas de fusión de sensores, dispositivos IoT y plataformas robóticas exige RTOS capaces de administrar grandes cantidades de tareas concurrentes manteniendo niveles mínimos de latencia y jitter. Por ello, resulta fundamental comprender cómo las estrategias de gestión de procesos y threads impactan en la capacidad de respuesta de estos sistemas [1], [7].

La pregunta central que guía el presente ensayo es: *¿Cuál es el impacto de la gestión de procesos y threads en la capacidad de respuesta de un Sistema Operativo de Tiempo Real?* Para responderla, se analizan los mecanismos internos de los RTOS modernos, se comparan resultados experimentales de diferentes plataformas y se discuten las tendencias actuales en el diseño de sistemas de tiempo real. Las fuentes utilizadas incluyen investigaciones recientes sobre optimización de planificación, comparaciones de rendimiento entre RTOS, estudios sobre comunicación en tiempo real, análisis de uso de memoria y evaluaciones de resiliencia ante fallos de hardware [1]–[9].

II. CONCEPTOS FUNDAMENTALES DE PROCESOS Y THREADS EN UN RTOS

II-A. Definición y Diferencias entre Procesos y Threads

Dentro de un RTOS, los procesos y threads representan las unidades básicas de ejecución. Un proceso es una instancia de un programa en ejecución que posee su propio espacio de memoria, recursos y contexto de ejecución. Un thread, en cambio, es una unidad de ejecución más ligera que comparte el espacio de memoria y los recursos del proceso al que pertenece. Esta diferencia fundamental tiene implicaciones directas sobre el rendimiento y la capacidad de respuesta del sistema [2], [6].

Debido a las limitaciones de recursos presentes en la mayoría de sistemas embebidos, los threads suelen ser utilizados con mayor frecuencia en los RTOS porque requieren menor consumo de memoria y presentan tiempos reducidos de cambio de contexto. Esta característica permite ejecutar múltiples tareas concurrentes manteniendo una respuesta rápida ante eventos externos. En sistemas como FreeRTOS, μ C/OS-II y VxWorks, las tareas son implementadas como threads que comparten el espacio de memoria del kernel, lo que facilita la comunicación entre ellas y reduce la sobrecarga del sistema [2], [9].

II-B. Estados de los Threads y Transiciones

La gestión eficiente de threads resulta especialmente importante porque cada thread posee estados de ejecución bien definidos. Los estados principales incluyen: *Listo* (ready), cuando el thread está preparado para ejecutarse pero espera acceso al procesador; *En ejecución* (running), cuando el thread ocupa activamente el procesador; *Bloqueado* (blocked), cuando el thread espera un recurso o evento externo; y *Suspendido* (suspended), cuando el thread ha sido detenido explícitamente por el sistema o por otra tarea [2], [6].

El tiempo requerido para cambiar entre estos estados constituye un factor determinante en la capacidad de respuesta del sistema. Cuanto menor sea el tiempo de transición entre estados, menor será la latencia experimentada por las aplicaciones críticas. Los RTOS modernos optimizan estas transiciones mediante estructuras de datos eficientes como listas enlazadas ordenadas por prioridad y tablas de bits de estado, que permiten identificar rápidamente el thread de mayor prioridad listo para ejecutarse [2], [9].

II-C. Mecanismos de Comunicación entre Threads

Los RTOS modernos implementan mecanismos avanzados de comunicación entre threads mediante semáforos, mutexes, colas de mensajes, buzones y eventos. Estos mecanismos permiten coordinar tareas concurrentes sin comprometer la predictibilidad temporal. La correcta utilización de estos recursos evita condiciones de carrera y facilita la sincronización eficiente entre componentes del sistema [6], [7].

Los semáforos son utilizados para controlar el acceso a recursos compartidos y para señalar eventos entre tareas. Los mutexes, por su parte, implementan exclusión mutua con soporte para herencia de prioridades, evitando el problema de inversión de prioridades. Las colas de mensajes permiten transferir datos entre tareas de forma segura y ordenada, mientras que los eventos permiten sincronizar múltiples tareas ante la ocurrencia de condiciones específicas [2], [4], [6].

III. PLANIFICACIÓN DE THREADS Y CAPACIDAD DE RESPUESTA

III-A. Algoritmos de Planificación en RTOS

Uno de los factores más importantes que determinan la capacidad de respuesta de un RTOS es el algoritmo de planificación utilizado. La mayoría de sistemas emplean planificación basada en prioridades, donde las tareas más críticas reciben

acceso preferencial al procesador. Este enfoque permite responder rápidamente ante eventos urgentes y minimizar los tiempos de espera de las tareas críticas [2], [6].

Entre los algoritmos más utilizados destacan los siguientes:

- **Fixed Priority Preemptive Scheduling (FPPS):** Asigna prioridades fijas a cada tarea y permite que una tarea de mayor prioridad interrumpa a una de menor prioridad en cualquier momento. Es el esquema más común en RTOS comerciales como VxWorks y QNX [2].
- **Rate Monotonic Scheduling (RMS):** Asigna prioridades inversamente proporcionales al período de cada tarea. Es óptimo para conjuntos de tareas periódicas con plazos iguales a sus períodos y ha sido ampliamente estudiado en la literatura de sistemas de tiempo real [6].
- **Deadline Monotonic Scheduling (DM):** Asigna prioridades inversamente proporcionales al plazo relativo de cada tarea. Es más general que RMS y permite manejar tareas con plazos menores a sus períodos [1].
- **Earliest Deadline First (EDF):** Selecciona en cada instante la tarea cuyo plazo absoluto sea más próximo. Es óptimo en términos de utilización del procesador pero puede presentar comportamientos impredecibles bajo condiciones de sobrecarga [2], [6].
- **Least Laxity First (LLF):** Selecciona la tarea con menor holgura temporal, definida como la diferencia entre el plazo y el tiempo de cómputo restante. Aunque teóricamente óptimo, su implementación práctica puede generar un número elevado de cambios de contexto [6].

III-B. Planificación Adaptativa y Optimización Dinámica

En aplicaciones modernas con cargas dinámicas, los algoritmos estáticos pueden presentar limitaciones para adaptarse a variaciones imprevistas en el comportamiento de las tareas. La investigación de Mishra propone un enfoque híbrido basado en Deadline Monotonic combinado con un mecanismo adaptativo denominado *Slack-Aware Priority Adjuster* (SAPA). Este mecanismo monitorea continuamente la holgura temporal disponible en el sistema y ajusta dinámicamente las prioridades de las tareas para optimizar la utilización del procesador sin comprometer el cumplimiento de plazos [1].

Los resultados experimentales de este enfoque muestran una reducción del 42 % en la latencia promedio y una mejora significativa en la estabilidad temporal del sistema. Además, el mecanismo SAPA logra reducir el jitter en un 38 % respecto a los esquemas tradicionales de prioridades fijas, lo que resulta especialmente beneficioso en aplicaciones de fusión de sensores donde la consistencia temporal es crítica [1].

III-C. Comparación de Rendimiento entre RTOS

Los estudios comparativos realizados entre diferentes RTOS evidencian diferencias significativas en términos de capacidad de respuesta. Merchant y Dedhia compararon sistemas como VxWorks, RTAI, RTLinux y μ C/OS-II, encontrando que los sistemas diseñados específicamente para tiempo real presentan latencias considerablemente menores que las soluciones basadas en Linux con parches de tiempo real [2].

Por su parte, Aroca y Caurin realizaron una comparación exhaustiva que incluyó parámetros como latencia de interrupción, tiempo de cambio de contexto, jitter y comportamiento bajo carga. Los resultados mostraron que VxWorks y QNX Neutrino destacan por ofrecer mecanismos avanzados de planificación que garantizan un comportamiento altamente determinista incluso bajo condiciones de carga elevadas, mientras que sistemas como FreeRTOS ofrecen un equilibrio adecuado entre funcionalidad y consumo de recursos para aplicaciones embebidas de menor complejidad [9].

IV. IMPACTO DEL CAMBIO DE CONTEXTO EN LA CAPACIDAD DE RESPUESTA

IV-A. Mecanismo y Sobrecarga del Cambio de Contexto

El cambio de contexto ocurre cuando el procesador interrumpe la ejecución de un thread para permitir que otro tome el control. Este mecanismo constituye uno de los componentes fundamentales de la multitarea, pero también representa una fuente importante de sobrecarga temporal que puede afectar significativamente la capacidad de respuesta del sistema [2], [8].

Cada cambio de contexto requiere almacenar el estado completo del thread actual, incluyendo el contador de programa, los registros del procesador, el puntero de pila y otros datos de contexto específicos de la arquitectura. Posteriormente, el sistema debe restaurar el estado del nuevo thread seleccionado por el planificador. Si este proceso consume demasiado tiempo, la capacidad de respuesta del sistema puede verse comprometida, especialmente en aplicaciones con requisitos temporales muy estrictos [2], [9].

Los estudios comparativos muestran que el tiempo de cambio de contexto varía significativamente entre diferentes RTOS. Sistemas como $\mu\text{C}/\text{OS-II}$ presentan tiempos de cambio de contexto del orden de microsegundos, mientras que soluciones basadas en Linux con parches de tiempo real pueden requerir decenas de microsegundos para completar la misma operación [2], [9].

IV-B. Implementación Hardware del Planificador: HartOS

Una evidencia clara de la importancia del cambio de contexto se observa en HartOS, un RTOS implementado parcialmente en hardware sobre FPGA (*Field-Programmable Gate Array*). Al trasladar funciones de planificación y administración de tareas directamente al hardware, HartOS logra reducir significativamente la sobrecarga asociada a los cambios de contexto y mejorar considerablemente la precisión temporal respecto a soluciones basadas únicamente en software [8].

La arquitectura de HartOS implementa en hardware un planificador de prioridades fijas con soporte para hasta 32 tareas concurrentes. El hardware se encarga de mantener la lista de tareas listas para ejecutarse y de seleccionar automáticamente la tarea de mayor prioridad, eliminando la necesidad de ejecutar código de planificación en el procesador principal. Los resultados experimentales demuestran que HartOS logra tiempos de cambio de contexto inferiores a $1 \mu\text{s}$, comparados con los $3.2 \mu\text{s}$ de $\mu\text{C}/\text{OS-II}$ en la misma plataforma [8].

IV-C. Frecuencia de Cambios de Contexto y Eficiencia

La frecuencia de cambios de contexto depende directamente de la estrategia de planificación utilizada y de las características de las tareas del sistema. Algoritmos con excesiva preempción pueden incrementar la sobrecarga del sistema, mientras que técnicas híbridas permiten mantener un equilibrio adecuado entre capacidad de respuesta y eficiencia computacional [1], [6].

En sistemas con muchas tareas de alta frecuencia, el tiempo dedicado a cambios de contexto puede representar una fracción significativa del tiempo total de procesamiento. Por esta razón, algunos RTOS implementan mecanismos de agrupación de tareas y ejecución cooperativa dentro de grupos de prioridad similar, reduciendo la frecuencia de cambios de contexto sin comprometer la capacidad de respuesta ante eventos críticos [4], [6].

V. LATENCIA, JITTER Y TIEMPO DE RESPUESTA

V-A. Definición y Clasificación de la Latencia

La latencia representa el tiempo transcurrido entre la ocurrencia de un evento y la respuesta correspondiente del sistema. En aplicaciones críticas, incluso pequeñas variaciones pueden afectar significativamente el comportamiento global del sistema. Por esta razón, la latencia constituye uno de los principales indicadores de desempeño de un RTOS [2], [9].

Existen diferentes tipos de latencia relevantes en un RTOS:

- **Latencia de interrupción:** Tiempo entre la ocurrencia de una interrupción hardware y el inicio de la ejecución del manejador de interrupción correspondiente [2].
- **Latencia de planificación:** Tiempo entre la finalización del manejador de interrupción y el inicio de la ejecución de la tarea de mayor prioridad que fue activada por dicha interrupción [6].
- **Latencia de comunicación:** Tiempo requerido para transferir datos entre tareas mediante mecanismos de comunicación como colas de mensajes o semáforos [3].
- **Latencia de acceso a memoria:** Tiempo necesario para acceder a datos almacenados en diferentes niveles de la jerarquía de memoria del sistema [4].

V-B. Comparación Experimental de Latencias

Los estudios comparativos realizados por Aroca y Caurin muestran diferencias significativas entre diversos sistemas operativos en términos de latencia de respuesta. Mientras Windows XP presentó una latencia promedio de $848 \mu\text{s}$ con picos de hasta varios milisegundos, sistemas especializados como $\mu\text{C}/\text{OS-II}$ registraron únicamente $3.2 \mu\text{s}$, evidenciando la importancia de un diseño orientado específicamente al tiempo real [9].

RTAI (*Real-Time Application Interface*) demostró ser uno de los sistemas con menor latencia entre las soluciones basadas en Linux, alcanzando valores de aproximadamente $15 \mu\text{s}$ en condiciones normales de operación. VxWorks, por su parte, presentó latencias consistentemente bajas del orden de $5\text{-}10 \mu\text{s}$, lo que lo convierte en una opción preferida para

aplicaciones industriales y aeroespaciales de alta exigencia [2], [9].

V-C. *Jitter y Predictibilidad Temporal*

El jitter se define como la variación temporal entre respuestas consecutivas del sistema ante eventos similares. Un bajo nivel de jitter garantiza que las tareas se ejecuten con una periodicidad estable y predecible, lo que resulta fundamental para aplicaciones de control en tiempo real donde la regularidad temporal es tan importante como la velocidad de respuesta [6], [9].

Sistemas como RTAI y VxWorks presentan valores de jitter considerablemente inferiores respecto a sistemas operativos convencionales. Los estudios muestran que RTAI puede mantener un jitter inferior a $10 \mu s$ bajo condiciones normales de carga, mientras que sistemas de propósito general como Linux estándar pueden presentar variaciones de varios milisegundos [9].

En aplicaciones modernas de fusión de sensores, donde múltiples fuentes de datos deben integrarse en tiempo real, la reducción de latencia y jitter resulta indispensable. Los resultados experimentales de Mishra muestran que mecanismos adaptativos de gestión de threads permiten mantener una respuesta consistente incluso bajo cargas dinámicas y eventos de ráfaga provenientes de sensores de alta frecuencia como IMUs, LiDARs y cámaras estereoscópicas [1].

VI. USO DE MEMORIA Y GESTIÓN DE THREADS

VI-A. *Restricciones de Memoria en Sistemas Embebidos*

La administración eficiente de memoria constituye otro factor clave en la capacidad de respuesta de los RTOS. Muchos sistemas embebidos operan con cantidades extremadamente reducidas de memoria RAM, que pueden variar desde unos pocos kilobytes hasta algunos megabytes. En este contexto, la forma en que se gestionan los stacks de los threads puede afectar significativamente el rendimiento global del sistema [4], [7].

Cada thread en un RTOS tradicional requiere un stack independiente para almacenar variables locales, parámetros de funciones y datos de contexto. El tamaño de este stack debe ser determinado en tiempo de diseño y generalmente se sobredimensiona para evitar desbordamientos. Cuando el sistema contiene muchas tareas, el consumo total de memoria de los stacks puede superar la capacidad disponible en el dispositivo [4].

VI-B. *Optimización del Uso de Memoria: Modelo de Stack Compartido*

Davis, Merriam y Tracey demostraron que los RTOS tradicionales que asignan un stack independiente a cada tarea generan un consumo elevado de memoria que puede resultar prohibitivo en plataformas con recursos limitados. Como alternativa, propusieron un modelo de ejecución basado en un único stack compartido y grupos de no preempción (*Non-Preemptive Groups*), logrando reducciones superiores al 75 %

en el uso de memoria sin comprometer la capacidad de respuesta del sistema [4].

En este modelo, las tareas que pertenecen al mismo grupo de no preempción comparten un único stack, ya que solo una de ellas puede estar en ejecución en un momento dado dentro del grupo. El planificador garantiza que las transiciones entre tareas del mismo grupo ocurran únicamente en puntos de sincronización predefinidos, eliminando la necesidad de mantener stacks separados para cada tarea [4].

La reducción del consumo de memoria no solo disminuye costos de hardware, sino que también permite ejecutar más tareas concurrentes dentro de plataformas limitadas. Esto resulta especialmente relevante en dispositivos IoT y sistemas embebidos de bajo consumo energético, donde los recursos disponibles son extremadamente restringidos y el costo del hardware es un factor crítico de diseño [4], [7].

VI-C. *Gestión Dinámica versus Estática de Memoria*

Los RTOS pueden implementar gestión dinámica o estática de memoria para los objetos del sistema. La gestión dinámica permite crear y destruir tareas, semáforos y colas en tiempo de ejecución, ofreciendo mayor flexibilidad pero introduciendo fragmentación de memoria y tiempos de asignación no deterministas. La gestión estática, en cambio, asigna todos los recursos en tiempo de compilación, garantizando tiempos de acceso deterministas pero limitando la flexibilidad del sistema [4], [5].

Para aplicaciones de tiempo real estricto, la gestión estática de memoria es generalmente preferida porque elimina la incertidumbre temporal asociada a las operaciones de asignación y liberación de memoria. Sistemas como $\mu C/OS-II$ y FreeRTOS ofrecen opciones para utilizar pools de memoria de tamaño fijo, que proporcionan tiempos de asignación deterministas y evitan la fragmentación [5], [6].

VII. COMUNICACIÓN Y SINCRONIZACIÓN ENTRE THREADS

VII-A. *Mecanismos de Sincronización y sus Implicaciones Temporales*

La capacidad de respuesta de un RTOS también depende de la eficiencia de los mecanismos de comunicación y sincronización entre tareas. Cuando múltiples threads comparten recursos comunes, pueden surgir problemas como condiciones de carrera, bloqueos mutuos e inversión de prioridades, todos los cuales pueden afectar negativamente la capacidad de respuesta del sistema [2], [6].

La inversión de prioridades ocurre cuando una tarea de alta prioridad debe esperar a que una tarea de baja prioridad libere un recurso compartido, mientras que tareas de prioridad intermedia continúan ejecutándose. Este fenómeno puede causar retrasos impredecibles en la respuesta de las tareas críticas y comprometer el cumplimiento de plazos temporales [4], [6].

VII-B. *Protocolos de Herencia de Prioridades*

Los mecanismos modernos de sincronización utilizan protocolos como *Priority Inheritance* (PI) y *Priority Ceiling* (PC)

para minimizar el problema de inversión de prioridades. El protocolo de herencia de prioridades eleva temporalmente la prioridad de la tarea que posee un recurso compartido al nivel de la tarea de mayor prioridad que espera dicho recurso, garantizando que el recurso sea liberado lo antes posible [4], [5].

El protocolo de techo de prioridades va más allá y asigna a cada recurso compartido una prioridad de techo igual a la máxima prioridad de todas las tareas que pueden acceder a dicho recurso. Cuando una tarea adquiere el recurso, su prioridad se eleva automáticamente al techo del recurso, evitando completamente la inversión de prioridades y los bloqueos mutuos [4], [6].

VII-C. Comunicación USART en Tiempo Real

En aplicaciones de comunicación serial, la gestión eficiente de threads tiene un impacto directo sobre el rendimiento del sistema. Habibie y colaboradores compararon el rendimiento de la comunicación USART (*Universal Synchronous/Asynchronous Receiver-Transmitter*) implementada mediante interrupciones tradicionales frente a implementaciones basadas en RTOS [3].

Los resultados mostraron que el comportamiento depende significativamente de la carga del sistema. Bajo cargas ligeras, los RTOS pueden ofrecer tiempos de comunicación menores gracias a la gestión organizada de las tareas de comunicación. Sin embargo, bajo cargas elevadas, las interrupciones tradicionales presentan ciertas ventajas en términos de velocidad de respuesta inmediata, aunque a costa de mayor complejidad en el código y menor capacidad para gestionar múltiples canales de comunicación simultáneos [3].

La principal ventaja de los RTOS en aplicaciones de comunicación radica en su capacidad para administrar simultáneamente múltiples tareas complejas manteniendo un comportamiento organizado y predecible. Esto resulta especialmente valioso en sistemas que deben gestionar comunicaciones con múltiples dispositivos periféricos mientras ejecutan tareas de procesamiento y control [3], [7].

VIII. RESILIENCIA Y ROBUSTEZ ANTE FALLOS

VIII-A. Influencia del Diseño del RTOS en la Resiliencia

Además de responder rápidamente, un RTOS debe ser capaz de mantener su funcionamiento ante errores de hardware y software. La resiliencia del sistema ante fallos es especialmente importante en aplicaciones críticas donde un fallo del sistema operativo puede tener consecuencias graves para la seguridad de personas o equipos [5].

Hoffmann, Dietrich y Lohmann demostraron que la estructura interna del sistema operativo influye directamente en su resiliencia frente a errores de memoria. Su investigación analizó cómo diferentes diseños de RTOS responden ante inyecciones de fallos en la memoria, simulando errores de hardware como los causados por radiación cósmica o interferencias electromagnéticas [5].

VIII-B. Diseño Estático versus Dinámico y Resiliencia

Los sistemas con asignación dinámica de objetos suelen depender intensamente de punteros y estructuras enlazadas para gestionar las listas de tareas, semáforos y colas. Esta dependencia aumenta la probabilidad de errores silenciosos cuando un fallo de memoria corrompe un puntero, ya que el sistema puede continuar ejecutándose con un comportamiento incorrecto sin detectar el error [5].

En contraste, los RTOS con diseño estático presentan estructuras más compactas y robustas, donde los objetos del sistema se almacenan en posiciones de memoria fijas y predecibles. Esta característica reduce significativamente la vulnerabilidad ante fallos de memoria, ya que la corrupción de un objeto tiene un impacto más localizado y predecible sobre el comportamiento del sistema [5].

Los resultados experimentales de Hoffmann y colaboradores muestran que los RTOS con diseño estático presentan tasas de detección de fallos significativamente mayores que los sistemas con diseño dinámico, lo que los convierte en opciones más adecuadas para aplicaciones de seguridad funcional donde la detección y recuperación ante fallos es crítica [5].

VIII-C. Impacto de la Resiliencia en la Capacidad de Respuesta

La resiliencia contribuye indirectamente a la capacidad de respuesta del sistema, ya que evita interrupciones inesperadas y garantiza la continuidad operativa de tareas críticas incluso en condiciones adversas. Un sistema que falla silenciosamente ante un error de memoria puede continuar ejecutándose pero con una capacidad de respuesta degradada o incorrecta, lo que puede ser más peligroso que un fallo explícito [5], [9].

En aplicaciones de seguridad funcional como sistemas de control de vuelo, frenos antibloqueo o equipos médicos, la resiliencia ante fallos es tan importante como la velocidad de respuesta. Por esta razón, los estándares de seguridad funcional como IEC 61508 y ISO 26262 establecen requisitos específicos sobre la capacidad de los RTOS para detectar y manejar errores de hardware y software [5], [6].

IX. RTOS EN EL CONTEXTO DEL INTERNET DE LAS COSAS

IX-A. Requisitos Específicos de los Sistemas IoT

El Internet de las Cosas representa uno de los dominios de aplicación más exigentes para los RTOS modernos. Los dispositivos IoT deben gestionar simultáneamente múltiples tareas como la adquisición de datos de sensores, el procesamiento local, la comunicación inalámbrica y la gestión de energía, todo ello con recursos computacionales y energéticos extremadamente limitados [7].

Milinković y colaboradores analizaron los criterios más importantes para seleccionar un RTOS adecuado para plataformas IoT. Entre los factores más relevantes destacan el tamaño del kernel, el consumo de energía, el soporte para protocolos de comunicación inalámbrica, la facilidad de desarrollo y la disponibilidad de herramientas de depuración [7].

IX-B. Comparación de RTOS para IoT

Los autores compararon sistemas como FreeRTOS, Contiki, TinyOS, RIOT y mbed OS, encontrando que cada uno presenta ventajas específicas según el tipo de aplicación. FreeRTOS destaca por su amplia compatibilidad con diferentes micro-controladores y su comunidad activa de desarrollo. Contiki y TinyOS están optimizados para redes de sensores inalámbricos con consumo energético mínimo. RIOT ofrece soporte completo para protocolos de Internet como IPv6 y CoAP, mientras que mbed OS proporciona una plataforma integrada con herramientas de desarrollo en la nube [7].

La gestión eficiente de threads en estos sistemas resulta fundamental para maximizar la vida útil de la batería mientras se mantiene la capacidad de respuesta ante eventos externos. Técnicas como el modo de bajo consumo entre tareas, la gestión dinámica de la frecuencia del procesador y la suspensión selectiva de periféricos permiten reducir significativamente el consumo energético sin comprometer la capacidad de respuesta [4], [7].

X. ANÁLISIS COMPARATIVO Y DISCUSIÓN

X-A. Síntesis de Resultados Experimentales

La revisión de la literatura permite identificar patrones claros en el impacto de la gestión de procesos y threads sobre la capacidad de respuesta de los RTOS. Los sistemas diseñados específicamente para tiempo real presentan consistentemente mejores resultados en términos de latencia, jitter y determinismo que las soluciones basadas en sistemas operativos de propósito general con extensiones de tiempo real [2], [9].

Los resultados experimentales muestran que la latencia de respuesta puede variar en varios órdenes de magnitud entre diferentes sistemas. Mientras $\mu\text{C}/\text{OS-II}$ y VxWorks presentan latencias del orden de microsegundos, sistemas como Windows XP o Linux estándar pueden presentar latencias de cientos de microsegundos o incluso milisegundos bajo condiciones de carga [9].

X-B. Factores Determinantes en la Capacidad de Respuesta

A partir del análisis de los estudios revisados, pueden identificarse los siguientes factores como los más determinantes en la capacidad de respuesta de un RTOS:

1. **Algoritmo de planificación:** La elección del algoritmo de planificación tiene el mayor impacto individual sobre la capacidad de respuesta. Los algoritmos adaptativos como SAPA demuestran mejoras significativas respecto a los esquemas estáticos tradicionales [1].
2. **Tiempo de cambio de contexto:** Un tiempo de cambio de contexto reducido permite responder más rápidamente ante eventos de alta prioridad. Las implementaciones hardware como HartOS demuestran que este parámetro puede reducirse significativamente mediante el uso de lógica dedicada [8].
3. **Gestión de memoria:** Una administración eficiente de la memoria de los stacks permite ejecutar más tareas concurrentes y reduce la probabilidad de fallos por desbordamiento [4].

4. **Mecanismos de sincronización:** Los protocolos de herencia de prioridades y techo de prioridades son esenciales para evitar la inversión de prioridades y garantizar la predictibilidad temporal [4], [6].

5. **Resiliencia ante fallos:** El diseño estático del RTOS mejora la robustez ante errores de hardware sin comprometer la capacidad de respuesta [5].

XI. CONCLUSIONES

La gestión de procesos y threads constituye uno de los elementos más determinantes en la capacidad de respuesta de un Sistema Operativo de Tiempo Real. Los mecanismos de planificación, sincronización, comunicación, cambio de contexto y administración de memoria influyen directamente sobre parámetros críticos como la latencia, el jitter y el tiempo de respuesta, que son fundamentales para garantizar el correcto funcionamiento de aplicaciones de tiempo real estricto [6], [9].

La revisión de la literatura demuestra que los RTOS modernos logran altos niveles de determinismo mediante algoritmos avanzados de planificación y mecanismos eficientes de gestión de recursos. Sistemas como VxWorks, QNX, RTAI y HartOS evidencian que una administración adecuada de threads permite responder rápidamente ante eventos externos incluso bajo condiciones de carga elevadas [2], [8], [9].

Las investigaciones recientes indican que las técnicas adaptativas basadas en monitoreo dinámico del sistema representan una evolución importante frente a los esquemas tradicionales de prioridades fijas. El mecanismo SAPA propuesto por Mishra demuestra que es posible reducir la latencia en un 42 % y el jitter en un 38 % mediante la gestión dinámica de prioridades, lo que resulta especialmente valioso en aplicaciones de fusión de sensores y sistemas autónomos [1].

La optimización del uso de memoria mediante modelos de stack compartido, como el propuesto por Davis y colaboradores, permite ejecutar sistemas complejos dentro de las restricciones de memoria de los dispositivos embebidos modernos, ampliando significativamente el rango de aplicaciones posibles sin incrementar los costos de hardware [4]. Asimismo, la resiliencia ante fallos de memoria, analizada por Hoffmann y colaboradores, demuestra que el diseño interno del RTOS tiene implicaciones directas sobre la confiabilidad del sistema en entornos adversos [5].

En el contexto del Internet de las Cosas, la selección adecuada del RTOS y la configuración óptima de sus mecanismos de gestión de threads resultan fundamentales para equilibrar la capacidad de respuesta con el consumo energético y las restricciones de recursos de los dispositivos IoT [7]. La comparación de sistemas como FreeRTOS, Contiki y RIOT evidencia que no existe una solución universal, sino que la elección debe realizarse en función de los requisitos específicos de cada aplicación.

En consecuencia, puede afirmarse que una gestión eficiente de procesos y threads no solo mejora el desempeño temporal de un RTOS, sino que constituye el factor principal que permite garantizar la confiabilidad, predictibilidad y seguridad

de los sistemas embebidos de tiempo real en aplicaciones modernas de alta exigencia computacional.

REFERENCIAS

- [1] D. N. Mishra, "Optimization of RTOS Scheduling for Latency-Sensitive Sensor Fusion Systems," *Electronics, Communications, and Computing Summit*, vol. 2, no. 3, pp. 89–97, Mar. 2026. [Online]. Available: <https://eccsubmit.com/index.php/congress/article/view/55>
- [2] S. Merchant and K. Dedhia, "Performance Comparison of RTOS," Columbia University, Technical Report, 2001. [Online]. Available: <https://www.cs.columbia.edu/~sedwards/classes/2001/w4995-02/reports/shamil-kalpen.pdf>
- [3] N. Habibie, M. R. Alhamidi, D. M. J. Purnomo, and M. F. Rachmadi, "Performance Comparison of USART Communication Between Real Time Operating System (RTOS) and Native Interrupt," *Jurnal Ilmu Komputer dan Informasi*, vol. 9, no. 1, p. 43, Feb. 2016, doi: <https://doi.org/10.21609/jiki.v9i1.365>.
- [4] R. Davis, N. Merriam, and N. Tracey, "How Embedded Applications using an RTOS can stay within On-chip Memory Limits," University of York, 2000. [Online]. Available: <https://www.cs.york.ac.uk/rts/static/papers/R:Davis:2000.pdf>
- [5] M. Hoffmann, C. Dietrich, and D. Lohmann, "Failure by Design: Influence of the RTOS Interface on Memory Fault Resilience," 2014.
- [6] P. Hambarde, R. Varma, and S. Jha, "The Survey of Real Time Operating System: RTOS," in *2014 International Conference on Electronic Systems, Signal Processing and Computing Technologies*, Jan. 2014, pp. 34–39, doi: <https://doi.org/10.1109/icesc.2014.15>.
- [7] A. Milinković, S. Milinković, and L. Lazić, "Choosing the right RTOS for IoT platform," *INFOTEH-JAHORINA*, vol. 14, 2015.
- [8] A. B. Lange, K. H. Andersen, U. P. Schultz, and A. S. Sørensen, "HartOS - a Hardware Implemented RTOS for Hard Real-time Applications," *IFAC Proceedings Volumes*, vol. 45, no. 7, pp. 207–213, 2012, doi: <https://doi.org/10.3182/20120523-3-cz-3015.00041>.
- [9] R. V. Aroca and G. Caurin, "A Real Time Operating Systems (RTOS) Comparison," *Workshop de Sistemas Operacionais (WSO)*, pp. 2441–2452, 2026. [Online]. Available: <https://sol.sbc.org.br/index.php/wso/article/view/36126>